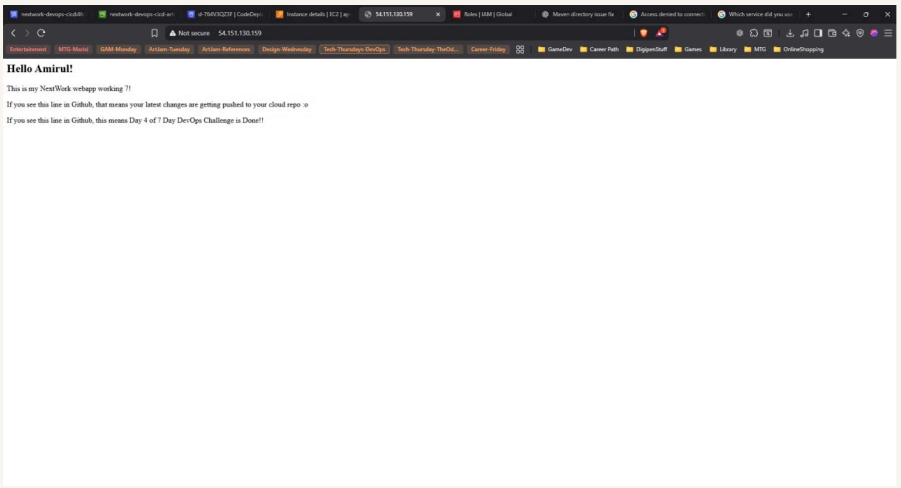
nextwork.org

Deploy a Web App with CodeDeploy

A Amirul Zaol-kefli



Introducing Today's Project!

In this project, I will demonstrate how to deploy code for webapp using AWS CodeDeploy. I'm doing this project to learn how to deploy code continuously for my webapp using AWS as part of the CI/CD project.

Key tools and concepts

Services I used were AWS CodeDeploy and CodeFormation. Key concepts I learnt include infrastructure as a code and code deployment, which is the second half of CI/CD.

Project reflection

This project took me approximately about 4 hours. The most challenging part was rebuilding the CodeBuild again from scratch. It was most rewarding to finally see my web app get deployed for once.

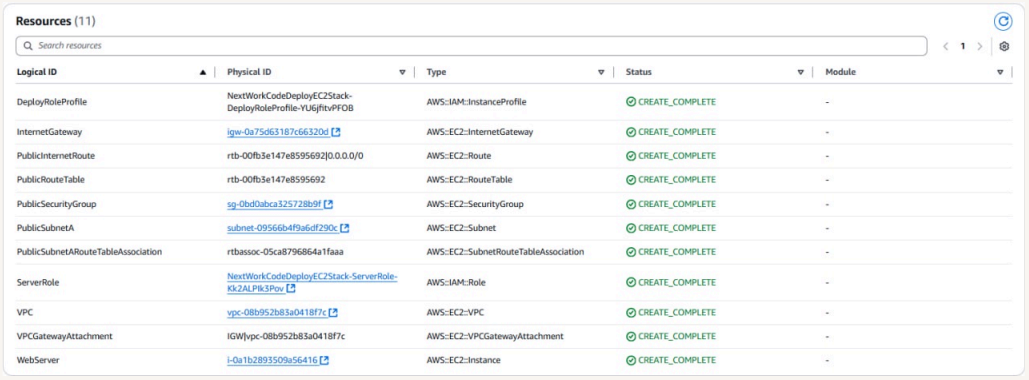
This project is part five of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project next Thursday.

Deployment Environment

To set up for CodeDeploy, I launched an EC2 instance and VPC because I want to control its network connectivity, IP addressing, and security settings, such as allowing it to be publicly accessible or isolated from the internet.

Instead of launching these resources manually, I used AWS CloudFormation. When I need to delete these resources I use AWS CloudFormation to determine how to delete these resources.

Other resources created in this template include VPC, Subnet, Route Tables, Internet Gateway, etc. They're also in the template because we are creating a complete, secure, and configurable infrastructure that can be easily replicated or modified.



Resources (11)

Search resources

Logical ID	Physical ID	Type	Status	Module
DeployRoleProfile	NextWorkCodeDeployEC2Stack-DeployRoleProfile-YUGjfitvPFOB	AWS::IAM::InstanceProfile	CREATE_COMPLETE	-
InternetGateway	igw-0a75d63187c66320d	AWS::EC2::InternetGateway	CREATE_COMPLETE	-
PublicInternetRoute	rtb-00fb3e147e8595692|0.0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE	-
PublicRouteTable	rtb-00fb3e147e8595692	AWS::EC2::RouteTable	CREATE_COMPLETE	-
PublicSecurityGroup	sg-0bd0abca325728b9f	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
PublicSubnetA	subnet-09566b4f9a6df290c	AWS::EC2::Subnet	CREATE_COMPLETE	-
PublicSubnetARouteTableAssociation	rtbassoc-05ca8796864a1faaa	AWS::EC2::SubnetRouteTableAssociation	CREATE_COMPLETE	-
ServerRole	NextWorkCodeDeployEC2Stack-ServerRole-Kk2ALPlk3Pov	AWS::IAM::Role	CREATE_COMPLETE	-
VPC	vpc-08b952b83a0418f7c	AWS::EC2::VPC	CREATE_COMPLETE	-
VPCGatewayAttachment	IGW|vpc-08b952b83a0418f7c	AWS::EC2::VPCGatewayAttachment	CREATE_COMPLETE	-
WebServer	i-0a1b2893509a56416	AWS::EC2::Instance	CREATE_COMPLETE	-

Deployment Scripts

Scripts are files with specific instructions in shell. To set up CodeDeploy, I also wrote scripts to deploy my code to a localhost.

The 'install_dependencies will tell the CodeDeploy to create special settings that let these programs to work together, making our website accessible to visitors on the internet.

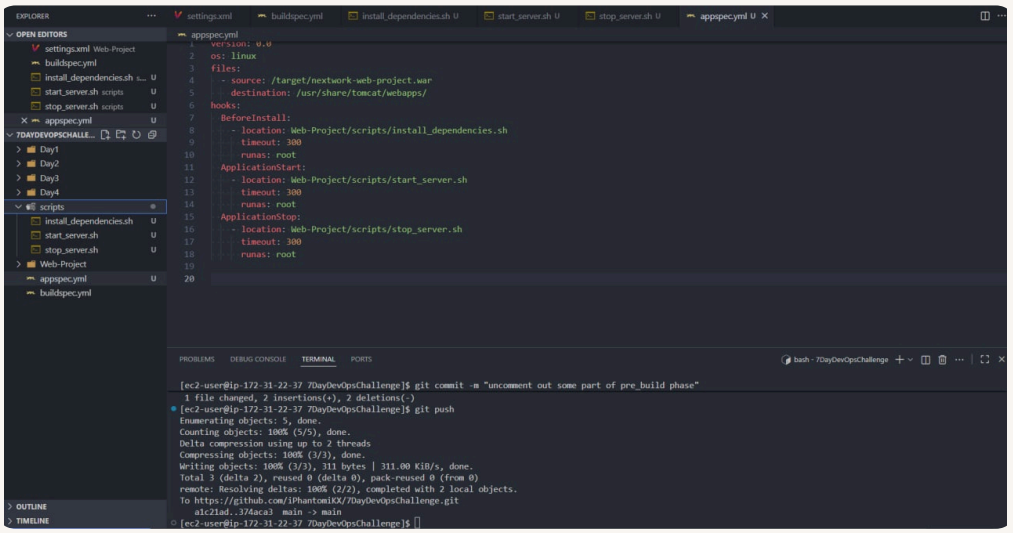
The start_server.sh will ensure our application is up and running and will stay that way even if the server restarts.

The stop_server.sh will stop the server from running by first checking if they're running and only attempts to stop the services if they are actually active.

appspec.yml

Then, I wrote an appspec.yml file to manually tell CodeDeploy what to do. The key sections in appspec.yml are the files and the hooks.

I also updated buildspec.yml because it needs to know how to build using appspec.yml and the new scripts.

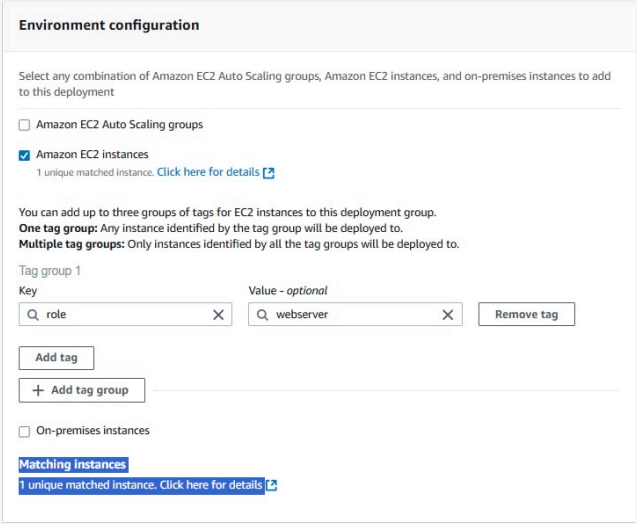


Setting Up CodeDeploy

A deployment group is a collection of EC2 instances that are grouped to deploy something together. A CodeDeploy application is like the main folder for your deployment project.

To set up a deployment group, you also need to create an IAM role to give CodeDeploy permission to access the CodeArtifact.

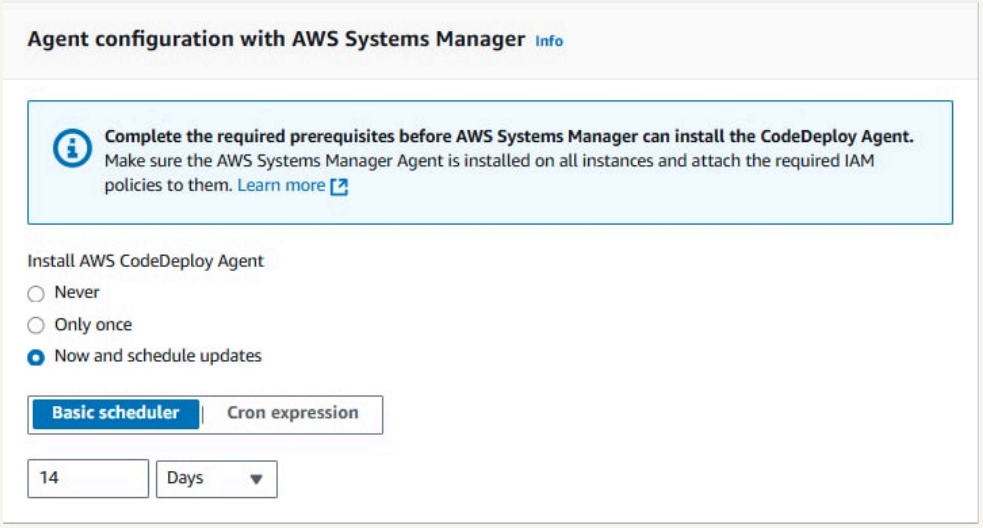
Tags are helpful for identifying which is the correct EC2 instance to use. I used the tag role to tag the current EC2 instance attached to my webapp and give it a value webserver.


Environment configuration
Select any combination of Amazon EC2 Auto Scaling groups, Amazon EC2 instances, and on-premises instances to add to this deployment
Amazon EC2 Auto Scaling groups
Amazon EC2 instances
1 unique matched instance. Click here for details
You can add up to three groups of tags for EC2 instances to this deployment group.
One tag group: Any instance identified by the tag group will be deployed to.
Multiple tag groups: Only instances identified by all the tag groups will be deployed to.
Tag group 1
Key
role
Value - optional
webserver
Remove tag
Add tag
Add tag group
On-premises instances
Matching instances
1 unique matched instance. Click here for details

Deployment configurations

Another key settings is the deployment configuration, which affects how quickly to roll out changes. I used CodeDeployDefault.AllAtOnce, so that all changes can be rolled out at the same time.

In order to connect to the WebServer, a CodeDeploy Agent is also set up to update every 14 days.

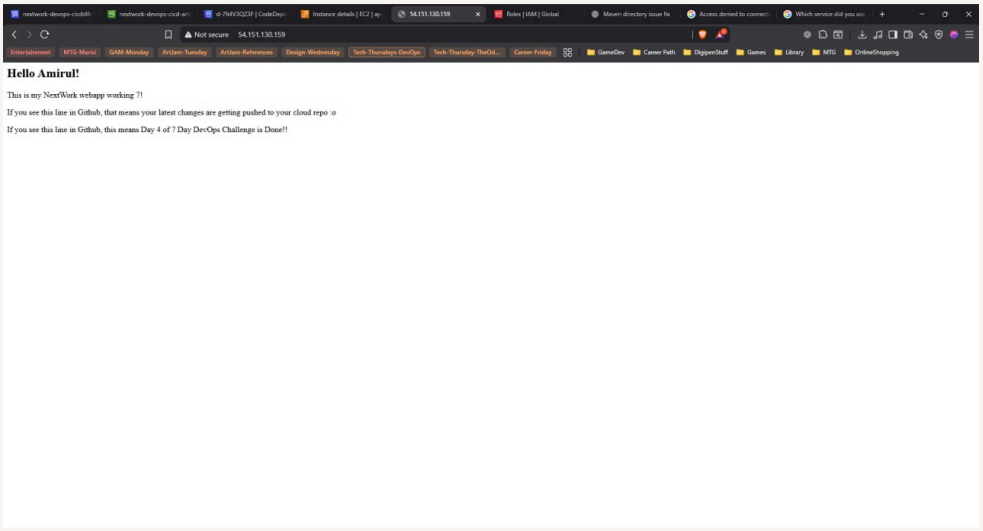


Success!

A CodeDeploy deployment is single update to your application. The difference to a deployment group is it determines the version of the app, where to deploy and how to deploy it.

I had to configure a revision location, which means it needs to know where the revisions are. My revision location was in my S3 Bucket.

To check that the deployment was a success, I visited the webserver app EC2 instance. I saw the words that were implemented in the index.jsp. However before I did that, I had to change the https to http in order to see it.


Not secure 54.151.130.159
Hello Amirul!
This is my NextWork webapp working ?!
If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o
If you see this line in Github, this means Day 4 of 7 Day DevOps Challenge is Done!!

The place to learn & showcase your skills

Check out nextwork.org for more projects